

Structure and Interpretation of Computer Programs (SICP)

Harold Abelson Gerald Jay Sussman Julie Sussman

Anotações do cap. 1.1.1 Expressions, pp. 7-10

Expressions that represents numbers can be combined with an expression representing a primitive procedure to form a combined expression that represents the result of the procedure with to those numbers.

`(+ 137 349) ; 486`

`(- 1000 335) ; 665`

`(* 5 99) ; 495`

`(/ 10 5) ; 2`

You can evaluate expressions that results in decimal numbers as well, without declaring a specific type.

`(+ 2.7 10) ; 12.7`

These expressions in lisp, formed by a list of expressions within parenthesis to denotate some operation, are called *combinations*. The result of these expressions are obtained by applying the operation specified by the operator (in the left) to the *arguments* (in the right) that are the the values of the operands.

The convention of placing the operator to the left is known by *prefix notation*. Despite being confusing, one of the advantages is the ability to declare a list of operands without any ambiguity regarding the operation being applied to them. Furthermore, you don't need to keep specifying the operation, as it is already defined on the left. Here are some examples:

`(+ 21 7 33 9 14) ; 84`

`(* 7 3 9) ; 189`

The second advantage of prefix notation is that it allows for the nesting of expressions in such a way that the entire expression combines into a cohesive whole.

In mathematical notation, this would be like: $(3 \times 5) + (10 - 5)$

`(+ (* 3 5) (- 10 5)) ; 20`

In mathematical notation, this would be like: $(3 \times ((2 \times 4) + (3 + 5))) + ((10 - 7) + 6)$

`(+ (* 3 (+ (* 2 4) (+ 3 5))) (+ (- 10 7) 6)) ; 57`

We can improve the way this expression is written by organizing it as follows:

```
(+ (* 3
    (+ (* 2 4)
        (+ 3 5)))
  (+
    (- 10 7)
    6)) ; which is 57 too
```